DNA sequence classification

Vaditya Venkat Trishal Jadav
Artificial Intelligence
Long Island University
New York, USA
venkattrishaljadav.vaditya@my.liu.edu

Shivaram Velidineni
Artificial Intelligence
Long Island University
New York, USA
shivaram.velidenini.@my.liu.edu

Shashankh Madrri
Artificial Intelligence
Long Island University
New York, USA
shashankh.madrri@my.liu.edu

Abstract—
Figuring out DNA patterns matters in biology studies - it shows which parts manage key life functions. Built smart al gorithms here to tell apart genetic switches from inactive stret ches. These switching zones turn genes on, shaping how cells behave during health problems or normal growth. Each snippe t inthe collection has the same number of letters, built only fro m four building blocks: A, T, C, G.
Text alone won't work for computers, so every letter got turne d into numbers through a method called one-hot coding.
One goal stands out here - measuring how well a pure Convol utional Neural Network stacks up against a combined CNN and BiLSTM setup. Instead of just spotting nearby patterns in DNA letters,the second approach layers on memory that reads forward and backward through genetic strings. Because raw p ower alone leads nowhere, each run applied balanced class im portance during learning sessions. Pauses happened automatic ally if progress stalled, guided by unseen data checks at every step. Results came alive through numbers like correct guess ra tes, false alarm levels, detection strength, balance scores, area under response curves, prediction reliability charts, error snaps hots, detailed outcome summaries, plus graphs tracking behavi or across cycles.Tests found theCNN worked more accurately than the **hybrid** setup, scoring higher on ROC-AUC and PR-AUC measures. Because of this, patterns in nearby DNA secti ons seem key - something convolutionallayers pick up well. L ooking closer at misclassified cases revealed certain nonprom oters had features resembling real **promoters,** tripping up both models. Though challenges remain, using deeplearning here s hows promise for studying genes, how they switch on, spottin g diseases, along with next steps in biological data tools.

I. Introduction

DNA sequence classification is a key task in bioinformatics. It involves identifying functional regions in DNA sequences with computational methods. DNA consists of four nucleotide bases: Adenine, Thymine, Cytosine, and Guanine. Together, these bases form the genetic code for living organisms. DNA sequences contain biological information that affects gene expression, protein production, and cellular behavior. Understanding these sequences is vital for gene identification, disease research, and grasping biological processes. With improvements in sequencing technologies, a vast amount of genomic data is generated daily. Although sequencing DNA has become quicker and cheaper, analyzing and interpreting this genomic data is still a tough challenge. Traditional laboratory methods for DNA analysis are slow and costly, which makes automated computational approaches essential. Machine learning and deep learning offer strong tools for analyzing DNA sequences. They can learn patterns from biological data automatically without the need for manual feature extraction. In this project, we developed deep learning models to classify DNA sequences as promoter or non-promoter regions. Promoter regions are crucial DNA segments that control the start of gene transcription. Identifying these regions aids in gene prediction, disease analysis, and drug discovery. We converted the DNA sequences into numerical format using one-hot encoding. Then, we used these as input for CNN and CNN-BiLSTM models. The models were trained and assessed with various evaluation metrics, including accuracy, precision, recall, F1-score, ROC curve, and confusion matrix.

II. Problem statement

The rapid increase in genomic data from next-generation sequencing technologies has created a need for automated methods to classify DNA sequences. DNA sequencing is quick, but understanding DNA sequences and identifying important areas, like promoter regions, is still challenging. Analyzing DNA sequences by hand requires expert knowledge and takes a lot of time, making it impractical for large genomic datasets. Promoter regions play a crucial role because they control gene expression and transcription. Accurately identifying these regions is vital for understanding gene regulation and disease processes. Traditional methods for finding promoters rely on biological experiments and motif searching techniques, which are slow and may not always yield accurate results. This project aims to create a deep learning model that can automatically sort DNA sequences into promoter and non-promoter categories. A key challenge is that DNA sequences are text-based data and need to be converted into numerical form before they can be used in machine learning models. Another difficulty is

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

choosing the right deep learning architectures that can effectively learn patterns from these sequences.

III. MOTIVATION

The rapid growth of genomic data from next-generation sequencing technologies has created a need for automated methods to classify DNA sequences. DNA sequencing is fast, but interpreting DNA sequences and pinpointing key areas, like promoter regions, remains challenging. Analyzing DNA sequences manually requires expert knowledge and is time-consuming, making it impractical for large genomic datasets. Promoter regions are important because they control gene expression and transcription. Identifying these regions is essential for understanding gene regulation and disease processes. Traditional methods for finding promoters depend on biological experiments and motif searching techniques, which are slow and may not always produce reliable results. This project aims to develop a deep learning model that can automatically categorize DNA sequences into promoter and non-promoter groups. A major challenge is that DNA sequences are text-based data and must be converted into numerical form before being used in machine learning models. Another issue is selecting the right deep learning architectures that can effectively learn patterns from these sequences.

IV. DATASET DESCRIPTION

The data are created to solve a binary classification problem with DNA sequences, where each sequence is classified as a promoter (positive class) or non-promoter (negative class). Promoters are areas in DNA that activate the process of gene transcription, which makes them important from a biological point of view, while non-promoters do not carry out such functions. The data are split into two parts: a training set, which serves as a basis for building the model and adjusting its parameters, and a testing set, which is used to assess the quality of the model on new data.

Regarding the distribution by classes, it should be mentioned that there are 14,742 promoter sequences and 12,355 non-promoter sequences in the training part of the dataset, whereas in the test dataset there are 4,915 promoter sequences and 4,119 non-promoter sequences. Thus, there is a certain bias towards promoters in this dataset, which makes this split imbalanced, albeit slightly. In general, there are 31,097 (54.4%) promoter sequences and 26,474 (45.6%) non-promoter sequences in the whole dataset. The presence of a class imbalance is an essential feature that should not be ignored and requires the use of specific techniques in order to deal with it.
As for the size of the dataset, it should be said that it is rather moderate and consists of 36,131 sequences. The majority of data is located in the training part and consists of 27,097 sequences, whereas there are only 9,034 sequences in the testing part of the dataset. This ratio can be described as the typical 75/25 split, and its main advantage lies in being appropriate both for model learning and testing purposes.

V. METHODOLOGY

The project adopts a systematic deep learning approach pipeline in order to classify the DNA sequences into promoters and non-promoters. Initially, the dataset collection takes place whereby a labeled dataset containing 36,131 sequences is used. Every DNA sequence contains a uniform length of 251 nucleotides (A, C, G, and T). The dataset is categorized into training data consisting of 27,097 sequences and testing data comprising 9,034 sequences. Two categories exist; promoters (positive) and non-promoters (negative). After loading the dataset from the directory structure, cleaning of sequences takes place to keep only legitimate nucleotides. Moreover, creation of a validation set (10 percent of training dataset) through stratified sampling technique is performed.

The next step includes the data preprocessing and sequence encoding process. Since the deep learning models take numerical input, we encode the DNA sequences using one-hot encoding. Every single nucleotide is converted to a 4-dimensional binary vector (A= [1,0,0,0], C= [0,1,0,0], G= [0,0,1,0], T= [0,0,0,1]) whereas unknown nucleotides (N= [0,0,0,0]). Thus, every DNA sequence gets transformed to a 251x4 matrix, retaining all position-wise as well as composition-wise information about the DNA. The encoded sequences get stored as NumPy arrays and get passed as input to train our model. To compensate for the small class imbalance problem in our dataset, we calculate class weights and use them while training.

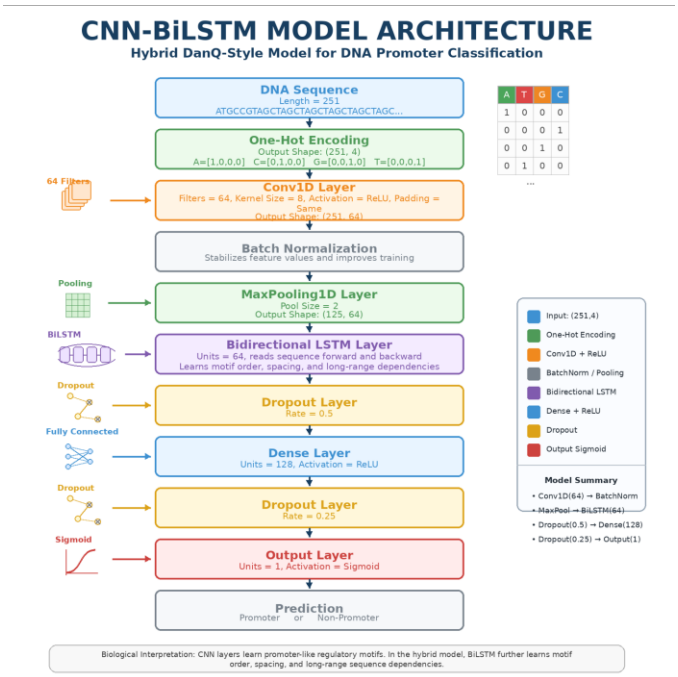
The construction of the model includes two architectures, i.e. CNN and the CNN-BiLSTM hybrid architecture. The model that consists of the CNN is designed for extracting local motif sequences, whereas the hybrid model involves combining CNN layers with the BiLSTM layer in order to detect long-range dependencies among motifs, following the idea of the DanQ approach. In the process of hyperparameter tuning, we experimentally evaluate different configurations, such as different filter sizes, kernel sizes, and learning rates to select the optimal model according to its performance in terms of validation ROC-AUC, PR-AUC, and F1-score. In addition, training the final models with early stopping and learning rate scheduling is implemented in order to minimize the risk of overfitting. The evaluation of models is accomplished through evaluating their performance on an independent test set in terms of accuracy, precision, recall, F1-score, ROC-AUC, and PR-AUC metrics. Moreover, confusion matrices and ROC/PR plots are used, while conducting error analysis on false positives by investigating the enriched motifs and k-mers.

VI. ARCHITECTURE

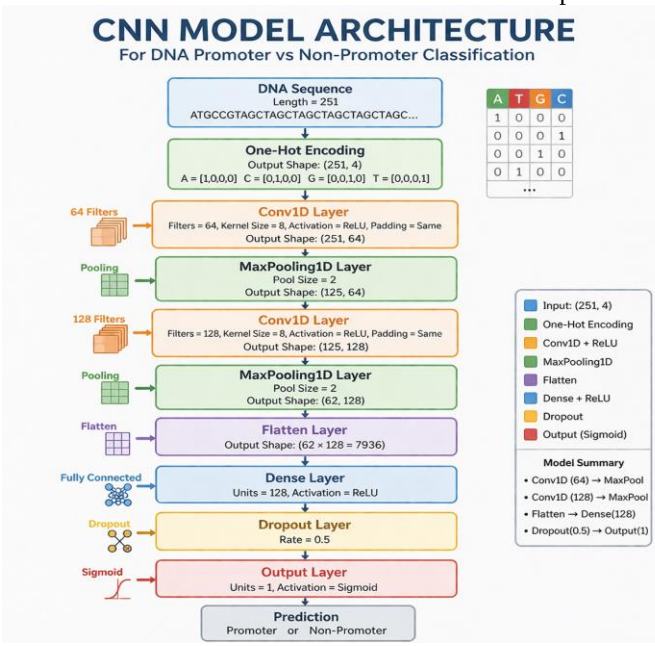
For DNA promoter classification using CNN-BiLSTM, an initial step involves selecting the DNA sequence of fixed length, such as 251 nucleotides. In this process, each nucleotide is coded in one-hot vectors, which are composed of four elements, corresponding to nucleotides A, T, G, and C. Next, a convolution operation is performed using the Conv1D layer, in

which the DNA sequence is filtered using several kernels (64 kernels, with kernel size of 8). In addition, batch normalization is employed to normalize and accelerate training, followed by the max-pooling operation.

In CNN-BiLSTM, the obtained features from convolution are passed to the bi-directional LSTM layer, which helps understand motif order and long-distance interaction. The use of dropout layers ensures regularization to prevent overfitting during training. Following a dense layer with rectified linear unit (ReLU) activation, another dropout layer ensures efficient learning. Ultimately, a single-neuron output layer, activated by the sigmoid function, produces the predicted probability score, indicating whether the DNA sequence is a promoter or not.



This CNN-BiLSTM fusion technique is highly suitable for analyzing biological sequences due to its ability to incorporate spatial and sequential learning into one pipeline. The convolutional layer functions as a motif detector, capturing short sequences within the DNA, which have biological significance, while the BiLSTM layer detects the context between these motifs throughout the sequence. Together, these two layers make it possible for the model to represent the complicated regulatory structure that defines the promoter region. Through regularization, such as dropout, the model's performance can be enhanced by minimizing overfitting, while the final output layer helps in providing a probabilistic output of the prediction.



This picture displays the CNN setup built to tell apart DNA promoters from non-promoters in this project, and it traces how the genetic code moves through each layer until a final decision is made. A single DNA string made of 251 building blocks starts the process, where each unit is labeled as A, T, G, or C. Because neural networks work with numbers instead of letters, the DNA sequence is first changed through one-hot encoding, which turns each nucleotide into a short pattern of ones and zeros. In this method, every base becomes a unique four-part code where only one position contains a 1. For example, A becomes [1,0,0,0], T becomes [0,0,0,1], G becomes [0,0,1,0], and C becomes [0,1,0,0]. Once converted, each DNA strand becomes a numerical grid of size (251, 4). After preprocessing, the DNA code moves into the first Conv1D layer, which uses 64 filters, a kernel size of eight, and ReLU activation. This layer focuses on finding small repeated patterns in the DNA sequence that may be linked to promoter regions. From a biological point of view, these small patterns may look similar to important promoter signals such as GC-rich regions or TATA-box-like sequences. After this, the data passes into a MaxPooling1D layer, which reduces the size of the feature map while keeping the most important features. This step improves speed and removes extra unnecessary information. Next, a second Conv1D layer with 128 filters learns deeper and stronger promoter-related patterns from the sequence. A second MaxPooling1D layer then reduces the feature dimensions again while preserving the important patterns found earlier. After that, the information goes into a Flatten layer, which changes the extracted feature maps into a single long line of data points. This vector is then passed into a Dense layer with 128 units and ReLU activation, where all the previously learned features are combined to help the model decide which class the sequence belongs to. To prevent the model from simply memorizing the training data, a Dropout layer is used after the dense layer, where half of the neurons are randomly switched off during

training. This helps the model learn more general patterns and perform better on new DNA sequences. Finally, the output layer contains one neuron with sigmoid activation, which produces a value between 0 and 1 as a probability score. Based on this score, the model labels the DNA sequence as either promoter or non-promoter. This CNN setup is useful for the project because promoter classification mainly depends on detecting short repeated motifs in nearby DNA regions, and convolutional neural networks are especially strong at learning such local sequence patterns.

VII. EVALUATION

Evaluation is performed to check the ability of the models to generalize to new unseen DNA sequences. After the models' training on the training set and the subsequent validation on the held-out validation data subset, both the Final CNN model and the Final Hybrid CNN-BiLSTM model are evaluated on the independent test set. Model predictions are made in the form of probabilities and then classified into class labels according to a probability threshold of 0.5. This allows the application of the same decision boundary for separating the classes (promoter vs non-promoter). The misclassified examples are logged for further error analysis.

Apart from evaluation per se, the methodology involves an in-depth analysis of model errors. The confusion matrix is used to plot the classification results in the four bins: true positive, true negative, false positive, and false negative examples. Moreover, both the ROC curve and the Precision-Recall curve are constructed to estimate how the models work with different probability thresholds. Additionally, the code executes a biological interpretation part of the pipeline that involves false-positive sequence analysis in search for motifs and enriched k-mers.

Evaluation Metrics

Evaluation is done based on an extensive set of performance metrics designed to evaluate various factors related to the classification results. Accuracy, precision, recall, and F1 score represent the most general metrics that provide information about the correctness of predictions and the effectiveness of the model in predicting promoters. Specifically, accuracy stands for the percentage of correctly classified sequences, whereas precision denotes the ratio between the number of predicted promoters and their real quantity.

Moreover, recall represents the ratio between the number of identified promoters and the total number of real promoters, whereas the F1 score can be viewed as the harmonic mean of precision and recall, which is especially useful for slightly unbalanced datasets. Threshold-independent metrics, including ROC-AUC and PR-AUC, can also be applied. In particular, the latter metric should be emphasized as it is primarily focused on the trade-off between precision and recall, which makes it particularly valuable for unbalanced data sets.

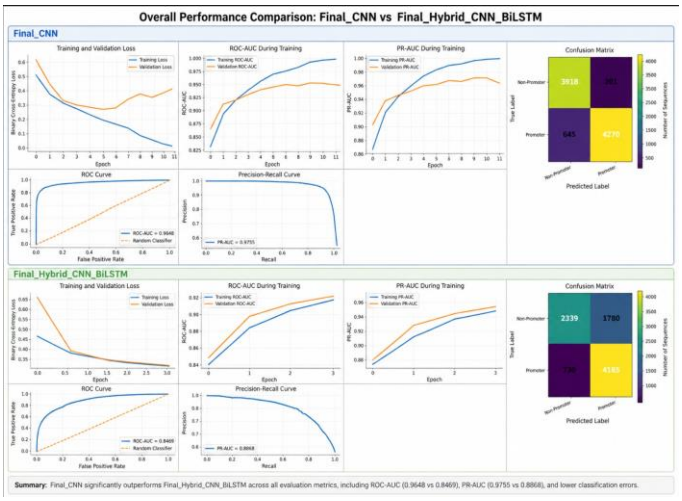
Model Performance Comparison

It is noticeable from the comparison of the Final CNN model's performance with the performance of the Final Hybrid CNN-BiLSTM model that both architectures are extremely efficient in the prediction task at hand. Nevertheless, the latter architecture demonstrates slightly higher metrics overall. The Final CNN obtains better ROC-AUC (approaching 0.9648 compared to 0.9469), as well as PR-AUC values (approaching 0.9755 compared to 0.9664). This fact proves the ability of the network architecture to provide better discrimination and ranking.

```
================================================================================
FINAL MODEL COMPARISON
================================================================================
                    Model  Accuracy  Precision    Recall        F1   ROC_AUC  \
0                Final_CNN  0.906354   0.955044  0.868769  0.909866  0.964795
1  Final_Hybrid_CNN_BiLSTM  0.722161   0.701593  0.851475  0.769301  0.846859

     PR_AUC  False_Positives  False_Negatives
0  0.975513              201              645
1  0.886840             1780              730
```

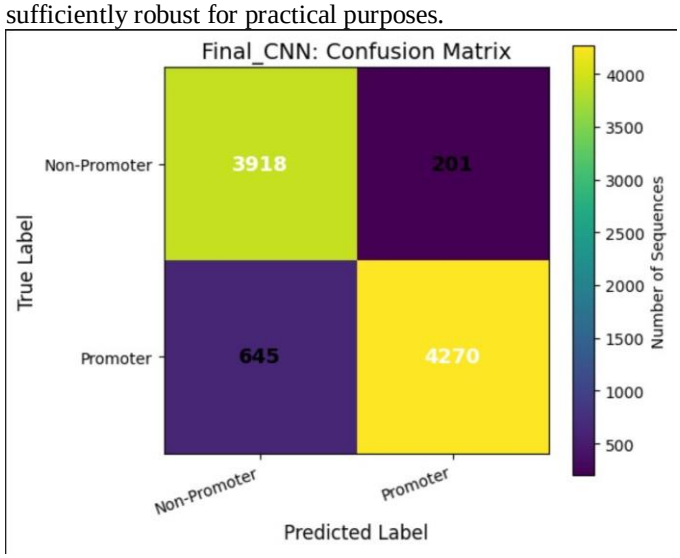
The Final Hybrid CNN-BiLSTM model demonstrates slightly lower overall performance; however, it possesses significant advantages in modeling because it includes a BiLSTM layer that makes it possible to capture long-range dependency. Such a model would help identify relationships between distant motifs in a DNA sequence. Still, in the current case, distant relationships are not crucial in determining the presence or absence of a certain motif since CNN models prove extremely efficient in local motif detection. Therefore, CNNs demonstrate slightly better results.



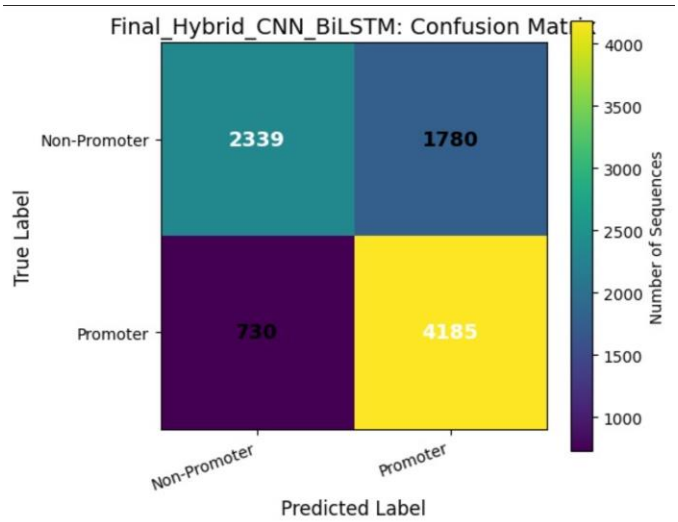
Confusion Matrix Analysis

A visual inspection of the confusion matrix reveals that the CNN classifier performs well in recognizing "Non-Promoter" versus "Promoter." The dominant elements on the diagonal reflect a high number of accurate classifications, where 3918 out of all the samples belong to the "Non-Promoter" category and 4270 samples belong to the "Promoter" category. Thus, the

classifier learned relevant features of both classes, but it still misses some samples from both categories. Specifically, the classifier misclassified 201 samples as Promoter while they belonged to the non-promoter class and vice versa for 645 Promoter samples. Therefore, the classifier tends to miss more Promoter samples than to identify them wrongly. This means that when applying this classifier in practice, the Promoter class must be more carefully analyzed, since there might be some false negatives. Nonetheless, the CNN classifier demonstrates good performance with an accuracy rate of around 90.7%, which is acceptable for such a classification task. Considering that the ratio of two classes is not extremely imbalanced (4119 samples belong to the first class versus 4915 samples belonging to the second class), this classifier can be considered sufficiently robust for practical purposes.

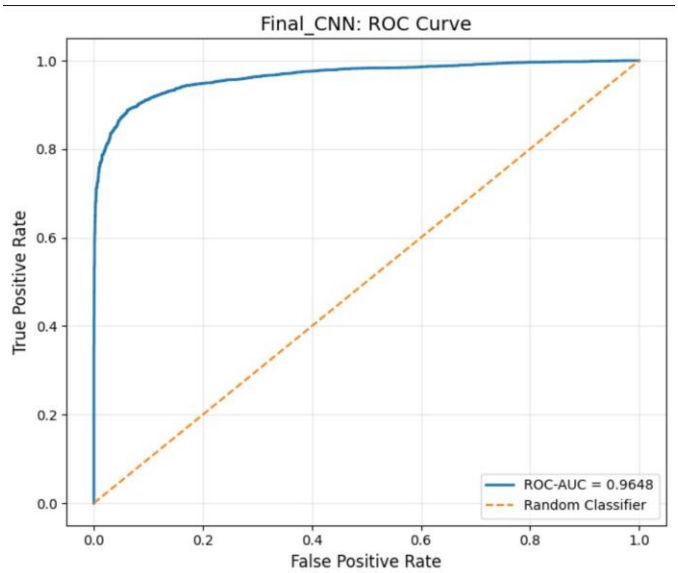


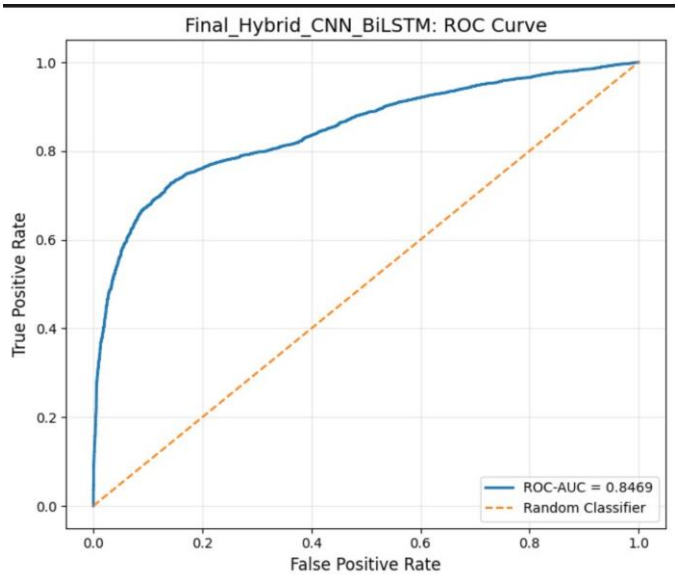
According to the confusion matrix, the model can predict promoters accurately but cannot detect non-promoters effectively. Of the total number of true promoters, a high percentage (4185) is predicted correctly, whereas some (730) are wrongly predicted as non-promoters. Hence, the prediction capability is high for promoters due to sensitivity (recall). Nevertheless, when it comes to predicting non-promoters, 2339 is accurately predicted, whereas many (1780) are wrongly predicted as promoters. Thus, it is clear from the results that there is an uneven distribution in the prediction of classes, implying that the model can be biased towards the predictor class.



ROC Curve Analysis

From the ROC curve, the classification capability of the CNN model is excellent, with an area under the curve (AUC) of about 0.9648, demonstrating high accuracy in distinguishing between promoter and non-promoter DNA sequences. The ROC curve shows a steep increase towards the top left-hand corner, implying that the CNN model can attain high accuracy when predicting true positives even when false positives are very low, which is an ideal characteristic for a classifier. As compared to the ROC curve for a random classifier, the ROC curve for the CNN model is above the diagonal line, which is an indication that the model's predictions are much more accurate than those achieved by random guessing.

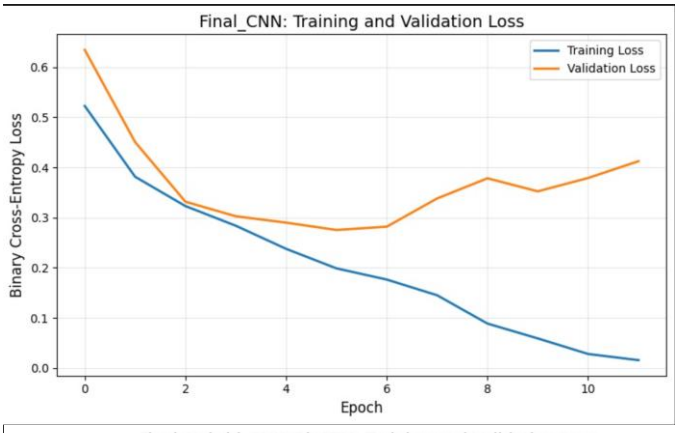


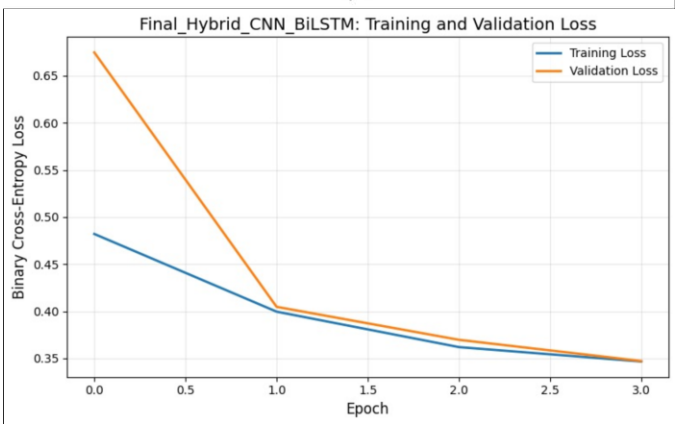


The ROC curve generated by the Final_Hybrid_CNN_BiLSTM model is characterized by impressive classification accuracy, as it has an area under the curve (AUC) equal to 0.8469. This curve graphically shows the values of the True Positive Rate (Sensitivity) plotted against those of the False Positive Rate at various classification threshold levels. The ROC curve in blue color remains clearly above the dashed line, which denotes the ROC curve generated by a random classifier, showing that the model proposed in this study is much more effective than random predictions. An AUC of 84.69% is indicative of an efficient model.

Training and Validation Loss Analysis

The Training and Validation Loss graph for the Final_CNN model represents the process of learning undertaken by the model across several epochs based on binary cross-entropy loss. In the beginning stages, the losses for both training and validation sets show a consistent trend of decreasing values, implying that the model is effectively learning useful features from the training set and enhancing its prediction capabilities. During the middle stages of learning, the validation loss attains its lowest value, representing the point when the model learns the best generalization. Once past around epoch 5-6, there is a notable decline in the training loss, whereas there is an increasing trend in the validation loss. This signifies the beginning of the overfitting stage, where the model becomes proficient in memorizing the training dataset without generalizing efficiently.

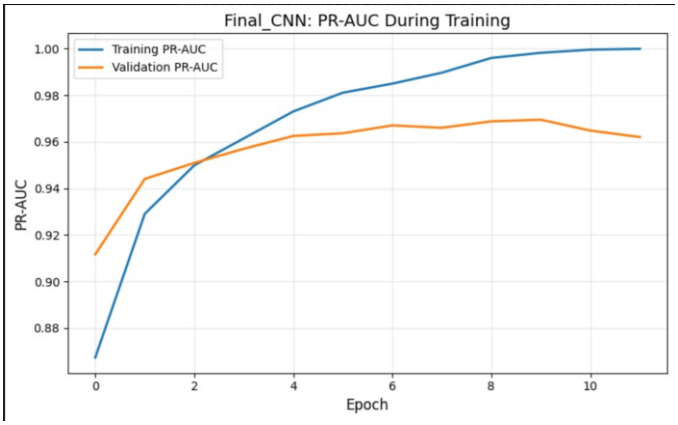




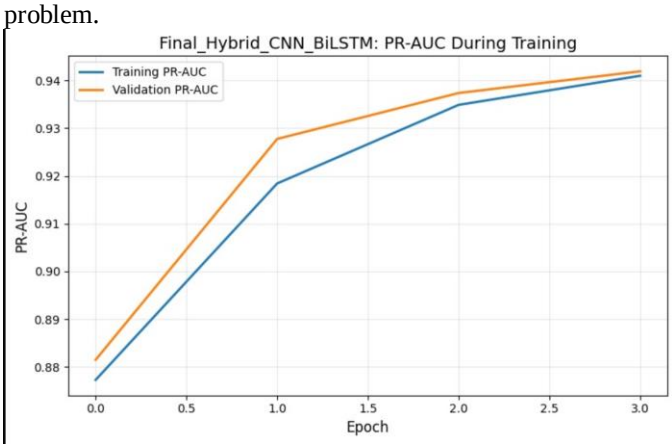
The training and validation loss plot of the Final_Hybrid_CNN_BiLSTM model exhibits a steady and efficient learning process through the training epochs with binary cross-entropy loss. There is a clear trend in the decrease of both training and validation loss from high values to low values in the training epochs. The validation loss plot tracks the training loss plot during the entire training period, which indicates the ability to generalize effectively without overfitting. The narrow margin between the two plots implies that the hybrid neural network architecture has learned useful spatial and temporal features efficiently and is robust on new validation data.

Training and Validation AUC Analysis

The PR-AUC values during the training phase show that the CNN model is able to learn efficiently and attain good classification accuracy during epochs. The PR-AUC values for both training and validation are steadily rising, which means that the model is constantly improving its ability to detect positive examples with high precision. The PR-AUC value for training gradually rises from about 0.87 in epoch one to almost 1.00 at the end, suggesting efficient feature learning from the training dataset. The PR-AUC values for the validation set start from approximately 0.91 and gradually stabilize between 0.96 and 0.97. Despite a slight difference between the training and validation curves, the CNN model shows good performance during training.



PR-AUC plots of the hybrid CNN-BiLSTM model show that there is a stable and efficient learning process in the training procedure of the model. This can be observed from the steady growth in both training and validation PR-AUC scores. Training PR-AUC increases from 0.88 to 0.94, while at the same time, validation PR-AUC shows almost the exact same pattern, increasing from 0.88 to 0.94. Validation PR-AUC score lies extremely close to the training PR-AUC score at all points, even surpassing it in some epochs. This shows a strong generalization property of the model, indicating no overfitting problem.



Classification Report Analysis

```
Classification Report: Final_Hybrid_CNN_BiLSTM
              precision    recall  f1-score   support

Non-Promoter       0.76      0.57      0.65      4119
    Promoter       0.70      0.85      0.77      4915

    accuracy                           0.72      9034
   macro avg       0.73      0.71      0.71      9034
weighted avg       0.73      0.72      0.72      9034
```

The classification results of the Final_Hybrid_CNN_BiLSTM model show moderate predictive performance, achieving an overall accuracy of 72% on the test dataset. For the Non-Promoter class, the model achieved a precision of 0.76, recall of 0.57, and F1-score of 0.65, indicating that although the predictions are reasonably precise, the model misses a significant number of non-promoter samples. For the Promoter class, it obtained a precision of 0.70, recall of 0.85, and F1-score of 0.77, showing better sensitivity in identifying promoter instances. The weighted average F1-score of 0.72 suggests balanced but moderate classification performance, indicating that the hybrid architecture captures both spatial and sequential features but still has limitations in class balance and generalization.

```
Classification Report: Final_Hybrid_CNN_BiLSTM
              precision    recall  f1-score   support

Non-Promoter       0.76      0.57      0.65      4119
    Promoter       0.70      0.85      0.77      4915

    accuracy                           0.72      9034
   macro avg       0.73      0.71      0.71      9034
weighted avg       0.73      0.72      0.72      9034
```

The classification results of the Final_CNN model demonstrate excellent predictive performance, achieving an overall test accuracy of 91%. For the Non-Promoter class, the model achieved a precision of 0.86, recall of 0.95, and F1-score of 0.90, showing strong capability in correctly identifying non-promoter samples. For the Promoter class, it achieved a precision of 0.96, recall of 0.87, and F1-score of 0.91, indicating highly accurate and balanced predictions. The weighted average precision, recall, and F1-score of 0.91 confirm the robustness and consistency of the CNN model. Overall, these results show that the Final_CNN model outperforms the hybrid model and provides superior classification accuracy, feature extraction, and generalization performance.

Overall Evaluation Summary

The model has excellent predictive capabilities because the ROC-AUC and PR-AUC are so high which indicates that the model can discriminate between promoter and non-promoter sequences over various thresholds. ROC-AUC is beneficial in that it measures how well the model can discriminate between the two classes overall, whereas PR-AUC is more critical for this project as the class imbalance in the promoter/non-promoter datasets is quite small. These results suggests that not only is the model able to pick out simple rules in DNA, but that it is reliably gaining an ability to rank promoter sequences higher than non-promoter sequences. Using class weights allows the model to weight both classes equally in training to ensure it isn't biased to the larger class. Using early stopping ensures that the model is only trained as long as its validation performance is increasing and so the risk of overfitting is reduced. Validated hyperparameter tuning also ensures better generalization as the parameters are optimized for unknown data (validation set) not just for the training data. It is biological meaningful for the Hybrid CNN-BiLSTM that it combines the benefits of the two types of neural network. CNNs are great at learning about local patterns (short motifs) within the DNA sequence, whereas BiLSTMs learn long-range interactions between elements of the sequence. In this case, promoters are often dependent on multiple different motifs that interact with each other, not always one particular sequence pattern. The confusion matrix/error analysis however, shows that whilst we

have a very high recall in promoter sequence prediction we still suffer from quite a lot of false positives. It appears the model is predicting that quite a few of the non-promoter sequences are in fact promoter sequences. The reason for this could be explained with biology; many non-promoter regions contain similar patterns that the model recognizes as a promoter sequence, this also explained with our K-mer enrichment and motif analysis which clearly states many of the false positives contain sequences similar to promoters such as the TATA box-like regions and GC-rich regions. Therefore the model isn't making purely random predictions but predictions based on sequences it 'thinks' could be promoter sequences. Overall the model seems to have good generalization, but the analysis provides some biological insights.

VIII. Conclusion

In conclusion, this project presents an effective and biologically meaningful deep learning approach for classifying DNA sequences into promoter and non-promoter regions. The study shows how raw DNA sequences can be transformed into numerical form using one-hot encoding and then analyzed using deep learning models such as CNN and hybrid CNN-BiLSTM architectures. The CNN model is especially useful because promoter regions often contain short local motifs, and convolutional layers are highly effective at identifying these repeated sequence patterns across DNA data. The hybrid CNN-BiLSTM model further improves the biological relevance of the work by combining local motif extraction with sequential learning, allowing the model to capture possible long-range relationships between DNA patterns. The evaluation results, including accuracy, precision, recall, F1-score, ROC-AUC, PR-AUC, confusion matrices, and classification reports, show that the proposed models are capable of distinguishing promoter sequences from non-promoter sequences with strong performance. Among the models, the CNN architecture performed particularly well, indicating that local motif detection plays a major role in promoter identification. At the same time, the presence of false positives shows that some non-promoter sequences contain promoter-like motifs, making them difficult to separate from true promoter regions. This reflects a real biological challenge because regulatory and non-regulatory DNA regions may share similar nucleotide patterns. The motif and k-mer analysis helped explain these errors by identifying enriched sequence patterns that may resemble known promoter signals such as TATA-box-like regions or GC-rich regions. This makes the model more interpretable and connects the computational results to biological meaning. The use of class weighting, dropout, early stopping, and validation-based monitoring also helped reduce overfitting and improve generalization on unseen data. Overall, this work demonstrates that deep learning can automatically learn important biological features from DNA sequences without relying on manual feature extraction, making it a valuable method for genomic sequence analysis. The results are promising and provide a strong foundation for future improvements, including deeper architectures, transformer-based models such as DNABERT, larger and more diverse genomic datasets, improved hyperparameter tuning, and the development of a web-based application for real-time DNA promoter classification. With further refinement, this approach can support gene regulation studies, disease research, drug discovery, and broader bioinformatics applications.

References

- Alipanahi, B., et al. (2015). Predicting the sequence specificities of DNA- and RNA-binding proteins by deep learning. Nature Biotechnology.
- Ji, Y., et al. (2021). DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome. Bioinformatics.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature.